

1. Website

- A collection of interlinked web pages, often with a common domain name, accessible via the internet.
- **Examples:** Static sites like blogs, portfolios, or informational sites. They are mostly used for viewing information and don't interact much with users beyond navigation and reading.

1. Static Website

- A website where the content is fixed and doesn't change dynamically. Each page is pre-built using HTML, CSS, and sometimes JavaScript, and is served to the user exactly as it is stored.

2. Dynamic Website

- A website where the content can change based on user interaction, input, or other variables. It relies on server-side technologies to generate content in real-time.

Internet

- A global system of interconnected computer networks that use the **TCP/IP** protocol suite to link billions of devices worldwide.
- **Examples of services:** World Wide Web (WWW), email, file sharing, online gaming.

Domain Name

- A human-readable address used to access websites on the internet. It maps to an IP address of the server where the website is hosted.
- **Structure:**
 - The domain name consists of two main parts: the name (e.g., **example**) and the top-level domain (TLD) (e.g., **.com**).

- **Example:** `www.example.com`
 - `example` is the domain name.
 - `.com` is the TLD.

WWW (World Wide Web)

- A system of interlinked hypertext documents accessed via the internet. It allows users to view and interact with web pages through web browsers.
- **Key Technologies:** HTML (for structuring pages), HTTP (for communication), and URLs (for identifying resources).
- **Important Note:** The WWW is a part of the internet, but the internet includes other services like email, FTP, etc.

Web Pages

- Documents on the web, typically written in HTML (HyperText Markup Language) and often styled with CSS (Cascading Style Sheets) and enhanced with JavaScript.
- **Types:**
 - **Static Web Page:** Content is fixed and the same for every visitor (e.g., a simple company "About Us" page).
 - **Dynamic Web Page:** Content is generated dynamically for each visitor based on factors like user interaction or database data (e.g., personalized dashboards, online store product pages).
- **Example:** A blog post, a product page, a contact form.

Web Application (Web App)

- A dynamic, interactive website that allows users to perform tasks or submit and manipulate data. Web apps are designed for user interaction and often rely on back-end logic.

Examples: Gmail, Facebook, Twitter, e-commerce sites, banking applications.

3. Web Server

- A server that stores, processes, and delivers web pages to clients (like a browser) based on requests.

- **Role:** The web server responds to HTTP requests from clients and sends back the requested web pages. It hosts both websites and web applications.
 - **Examples:** Apache, Nginx, Microsoft IIS.
-

4. Application Server

- A server designed to host applications and provide business logic to clients, typically interacting with a web server.
 - **Role:** It processes complex application-level tasks, such as database access, transactions, and API handling.
 - **Examples:** JBoss, GlassFish, WebLogic, IBM WebSphere.
-

5. Client

- The device or program (usually a web browser) used by a user to access a web server or web application.
 - **Examples:** Browsers like Chrome, Firefox, Safari, or any device that makes HTTP requests to a server.
-

6. Server

- A computer system that provides resources, data, services, or programs to clients over a network.
 - **Role:** Responds to client requests, processes the logic, and sends back the appropriate response.
 - **Examples:** Web servers, application servers, database servers.
-

7. Request

- An HTTP message sent from the client to the server asking for specific resources, such as a web page or data from an API.
- **Components:** HTTP method (GET, POST, etc.), headers, and optional body (in case of POST, PUT).

- **Example:** When you type a URL in a browser, it sends a request to the server to load the page.
-

8. Response

- The server's reply to a client's request, usually containing the requested resource (HTML, JSON, images, etc.) and a status code indicating the success or failure of the request.
 - **Components:** Status code (e.g., 200 OK, 404 Not Found), headers, and body (HTML, JSON, etc.).
-

9. URL (Uniform Resource Locator)

- The address used to identify a resource on the internet.

Structure:

`protocol://hostname:port/path?query#fragment`

Example:

`https://www.example.com:8080/homepage?search=query#section1`

- - **protocol:** Communication protocol (e.g., `http`, `https`).
 - **hostname:** Domain name or IP address (e.g., `www.example.com`).
 - **port:** Optional, if not the default (e.g., `8080`).
 - **path:** Specific page or resource (e.g., `/homepage`).

- **query**: Optional query parameters (e.g., `search=query`).
 - **fragment**: Optional section in a page (e.g., `#section1`).
-

10. Protocol

- A set of rules or procedures for transmitting data between devices over a network. In web development, HTTP (Hypertext Transfer Protocol) and HTTPS (secure version of HTTP) are the most common.
 - **Examples**: HTTP, HTTPS, FTP, SMTP.
-

11. Structure of URL

The structure of a URL can be broken down into the following components:

`protocol://username:password@hostname:port/path?query=param#fragment`

- **Protocol**: Communication protocol like `http` or `https`.
 - **Username/Password** (optional): Sometimes included for authentication.
 - **Hostname**: The domain name or IP address of the server.
 - **Port**: An optional port number, typically default for HTTP (80) or HTTPS (443).
 - **Path**: Path to the specific resource.
 - **Query Parameters**: Optional data sent to the server as part of the request.
 - **Fragment**: A reference to a section within a page.
-

12. HTTP Methods

- HTTP methods define the action to be performed for a given resource. Common HTTP methods are:
 - **GET**: Request data from a resource (e.g., loading a web page).
 - **POST**: Submit data to the server to create/update a resource (e.g., submitting a form).
 - **PUT**: Update an existing resource.
 - **DELETE**: Delete a specified resource.
 - **PATCH**: Partially update a resource.
 - **HEAD**: Similar to GET but only retrieves headers (no body).
 - **OPTIONS**: Returns the supported HTTP methods of the resource.
 - **CONNECT**: Establish a network connection to a resource.
 - **TRACE**: Used for diagnostic purposes.
-

13. HTTP (Hypertext Transfer Protocol)

- The protocol used to transfer web pages on the internet. It defines the structure of messages between clients and servers.
- **Features:**
 - Stateless: Each request is independent; the server does not retain any information about previous requests.
 - Methods like GET, POST, PUT, DELETE, etc., dictate what kind of operation should be performed on the resource.

What is Servlet

A servlet is a Java program that runs within a Web server. Servlets receive and response to requests from Web clients, usually across HTTP, the HyperText Transfer Protocol.

Servlets are built from two packages:

- `javax.servlet`
- `javax.servlet.http`

Servlet

1. Servlet is interface in `javax.servlet` packages

2. To create a servlet we have inherits the servlet interface
3. Servlet interface has 5 abstract method,we have override in servlet class

Methods In Servlet

public void init(ServletConfig config)

initializes the servlet. It is the life cycle method of servlet and invoked by the web container only once.

public void service(ServletRequest request, ServletResponse response)

responds to the incoming request. It is invoked at each request by the web container.

public void destroy()

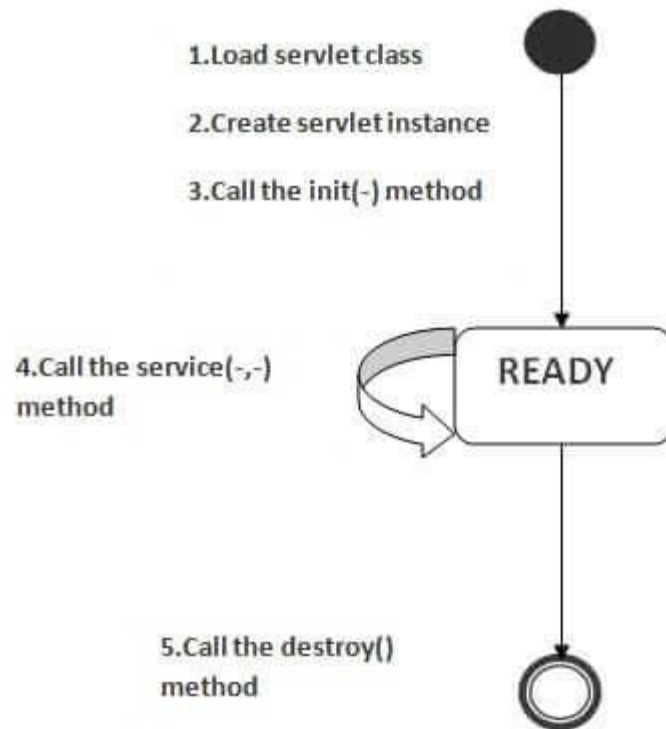
is invoked only once and indicates that the servlet is being destroyed.

public ServletConfig getServletConfig()

returns the object of ServletConfig.

public String getServletInfo()

returns information about servlet such as writer, copyright, version etc.



Servlet Life Cycle

The life cycle of a servlet refers to the series of stages that a servlet goes through from its instantiation to its destruction.

Loading:

- When a servlet container starts or when a request is received for the servlet for the first time, the container loads the servlet class.

Instantiation:

- After loading, the servlet container creates an instance of the servlet class using the new keyword.

Initialization:

- Once the servlet is instantiated, the `init()` method is called. This method is used to perform any necessary initialization tasks.

Request Handling:

- The servlet is now ready to handle client requests. The `service()` method is called by the container to process each client request.

Destroying:

- When the servlet container is shutting down or decides that the servlet is no longer needed, it calls the `destroy()` method.

Introduction To Springs

- Application framework
- Developed **by Rod Johnson** in 2004.
- First name was Interface21
- Spring was introduce to overcome drawback of EJB Application
- Spring is Lightweight application
- Make code easy
- Spring help to achieve loose coupling

Spring makes programming Java quicker, easier, and safer for everybody. It's focus on speed, simplicity, and productivity has made it the world's most popular Java framework.

Whether you're building secure, reactive, cloud-based microservices for the web, or complex streaming data flows for the enterprise, Spring has the tools to help..

Born as alternative to EJB in 2000 ,the spring framework quickly over took its opponent with its simplicity, variety of features, and its third-party library integrations.

It is so popular, that its main competitor quit the race when Oracle stopped the evolution of Java EE 8, and the community took over its maintenance via Jakarta EE.

The main reason of Spring framework success is, it regularly introduces features/projects based on the latest market trends, needs of the Dev community. For ex: Spring-Boot

Spring is open source. It has a large and active community that provides continuous feedback based on a diverse range of real-world use cases.

Coupling

- In object-oriented design **coupling** refers to the degree of direct knowledge that one class has of another. It's a measure of how interconnected classes or modules are, which affects how easily the code can be modified or maintained.

There are two types of coupling:

1.Tight Coupling

When one class relies heavily on the implementation details of another class, the classes are tightly coupled.

If class A directly creates an object of class B and calls its methods, class A is tightly coupled to class B.

```
class B {  
    public void doSomething() {  
        System.out.println("Doing something in B");  
    }  
}  
  
class A {  
    B b = new B(); // Class A depends on Class B (tight coupling)  
  
    public void doWork() {  
        b.doSomething();  
    }  
}
```

Disadvantages:

- Makes code harder to modify or extend because changing class B might require changes in class A.
- Limits code reusability.
- Reduces flexibility in replacing or refactoring the implementation of classes.

2.Loose Coupling

When classes have a minimal amount of dependency or knowledge of each other, they are loosely coupled.

Advantages

- Easier to modify, maintain, and extend code.
- Increases reusability and flexibility.
- Reduces the ripple effect when changes occur in one class.

```
interface Worker {
    void doWork();
}

class B implements Worker {
    public void doWork() {
        System.out.println("Doing work in B");
    }
}

class A {
    Worker worker;

    public A(Worker worker) { // Class A depends on the Worker interface, not a
        specific implementation (loose coupling)
        this.worker = worker;
    }

    public void execute() {
        worker.doWork();
    }
}
```

Enterprise JavaBeans (EJB) is a server-side software component architecture for Java EE (now Jakarta EE) used to build scalable, distributed, and transactional business applications. EJBs encapsulate business logic, making it easier to develop enterprise applications by handling aspects like transaction management, concurrency, security, and remoting

EJB vs. Spring Framework:

While EJB was a dominant enterprise framework for a long time, many developers prefer the **Spring Framework** due to its simplicity, flexibility, and lighter resource requirements. Spring provides an alternative for EJB

by offering similar features (transaction management, security, etc.) without requiring a full Java EE application server.

Spring Version

Version	Date	Notes
0.9	2003	
1.0	March 24, 2004	First production release.
2.0	2006	
3.0	2009	
4.0	2013	
5.0	2017	
6.0	November 22, 2022	

JEE Evolution

Java enterprise platform history

Platform version	Release	Specification	Java SE Support	Important Changes
Jakarta EE 11	Planned for June/July 2024	11	Java SE 21	
Jakarta EE 10	2022-09-13 ^[9]	10	Java SE 17 Java SE 11	Removal of deprecated items in Servlet, Faces, CDI and EJB (Entity Beans and Embeddable Container). CDI-Build Time.
Jakarta EE 9.1	2021-05-25 ^[10]	9.1	Java SE 11 Java SE 8	JDK 11 support
Jakarta EE 9	2020-12-08 ^[11]	9	Java SE 8	API namespace move from <code>javax</code> to <code>jakarta</code>
Jakarta EE 8	2019-09-10 ^[12]	8	Java SE 8	Full compatibility with Java EE 8
Java EE 8	2017-08-31	JSR 366	Java SE 8	HTTP/2 and CDI based Security
Java EE 7	2013-05-28	JSR 342	Java SE 7	WebSocket , JSON and HTML5 support
Java EE 6	2009-12-10	JSR 316	Java SE 6	CDI managed Beans and REST
Java EE 5	2006-05-11	JSR 244	Java SE 5	Java annotations
J2EE 1.4	2003-11-11	JSR 151	J2SE 1.4	WS-I interoperable web services ^[13]
J2EE 1.3	2001-09-24	JSR 58	J2SE 1.3	Java connector architecture ^[14]
J2EE 1.2	1999-12-17	1.2	J2SE 1.2	Initial specification release

Spring Introduction

Spring makes programming Java quicker, easier, and safer for everybody. It's focus on speed, simplicity, and productivity has made it the world's most popular Java framework.

Whether you're building secure, reactive, cloud-based microservices for the web, or complex streaming data flows for the enterprise, Spring has the tools to help..

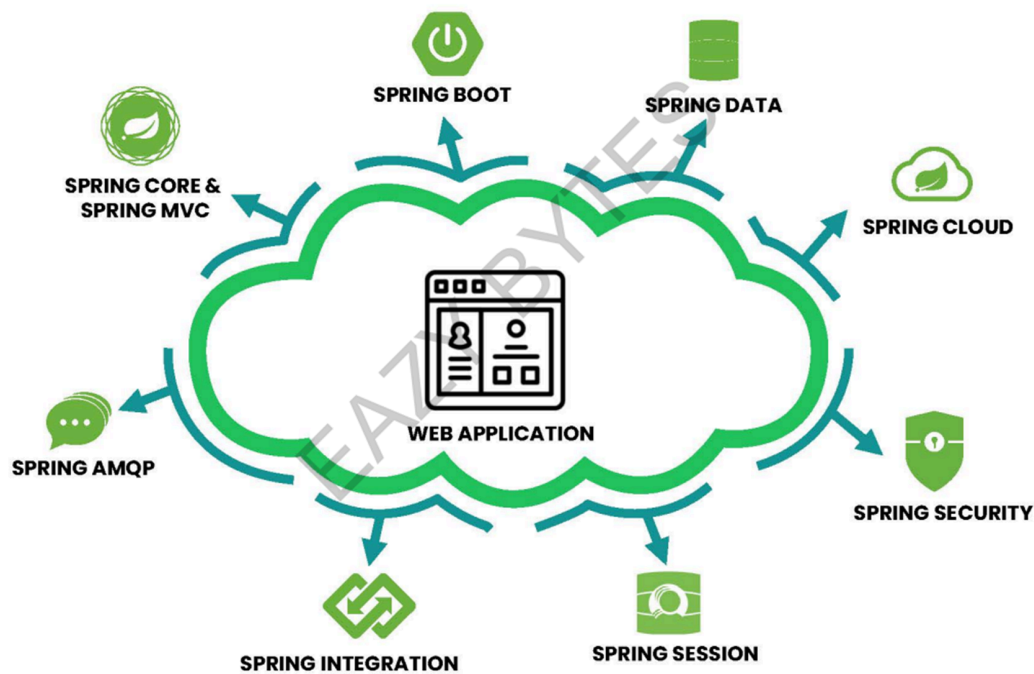
Born as alternative to EJB in 2000 ,the spring framework quickly over took its opponent with its simplicity, variety of features, and its third-party library integrations.

It is so popular, that its main competitor quit the race when Oracle stopped the evolution of Java EE 8, and the community took over its maintenance via Jakarta EE.

The main reason of Spring framework success is, it regularly introduces features/projects based on the latest market trends, needs of the Dev community. For ex: Spring-Boot

Spring is open source. It has a large and active community that provides continuous feedback based on a diverse range of real-world use cases.

Spring Modules



Spring Core Container

- The core of the Spring Framework. It contains the **Inversion of Control (IoC)** and **Dependency Injection (DI)** features, which are the foundation of the framework.

Spring AOP (Aspect-Oriented Programming)

- Provides support for separating cross-cutting concerns (such as logging, transaction management, and security) from the business logic. AOP allows for the modularization of concerns that cut across multiple layers of an application.

Spring Data Access / ORM

- Provides integration with various data access technologies and frameworks.
- **Modules:**
 - **JDBC Module:** Simplifies the use of JDBC (Java Database Connectivity) by removing the boilerplate code (like handling exceptions and managing resources).
 - **ORM Module:** Offers integration with ORM frameworks like Hibernate, JPA, iBatis.
 - **Spring Data:** Part of the broader data access support, simplifying access to NoSQL databases (e.g., MongoDB, Redis) and relational databases.

Spring Web

- Focuses on building web-based applications, including RESTful web services.
- **Modules:**
 - **Spring MVC (Model-View-Controller):** A framework for building web applications using the MVC design pattern. It integrates with various view technologies (e.g., JSP, Thymeleaf).
 - **Spring WebFlux:** A reactive programming model for building asynchronous, non-blocking web applications. It's suitable for large-scale applications that need to handle a large number of concurrent users.

Spring Security

- Provides security services for Spring applications, including authentication, authorization, and protection against common vulnerabilities like CSRF and SQL injection.
- **Features:**
 - Declarative security with annotations and configuration.
 - Comprehensive support for integrating with OAuth, LDAP and other security protocols.

Spring Boot

- A project that simplifies Spring application development by providing a convention-over-configuration approach. It allows for rapid development with embedded servers and minimal setup.
- **Features:**
 - Pre-configured dependencies and settings to quickly get a Spring application up and running.
 - Built-in support for embedded servers like Tomcat and Jetty.
 - Auto-configuration of Spring components.
 - Simplified creation of microservices.

Spring Cloud

- A suite of tools to build cloud-native microservices that can run in distributed environments.
- **Modules:**
 - **Spring Cloud Config:** Centralized configuration management for distributed systems.
 - **Spring Cloud Netflix:** Tools for service discovery, circuit breakers, and load balancing (using Netflix OSS components like Eureka, Hystrix).
 - **Spring Cloud Gateway:** API gateway for routing requests and managing services.

Spring Batch

- Provides reusable functions for processing large volumes of data, such as reading and writing to databases, handling large file processing, etc.
- **Features:**
 - Declarative I/O operations.
 - Built-in retry and skip functionality for batch jobs.
 - Support for chunk-based and task-based processing.

Spring Integration

- **Definition:** Helps integrate systems through messaging and provides support for enterprise integration patterns (EIP).
- **Features:**
 - Messaging framework to handle system-to-system communication.
 - Provides adapters for different protocols (JMS, FTP, SMTP, etc.).

Spring AMQP

- Provides support for developing applications that use message queues, specifically for **AMQP** (Advanced Message Queuing Protocol) with RabbitMQ.
- **Features:**
 - Easy integration with RabbitMQ.
 - Message handling and routing support.

Spring Messaging

- Provides messaging support using message brokers, facilitating asynchronous communication between components.
- **Features:**
 - Support for message-based communication.

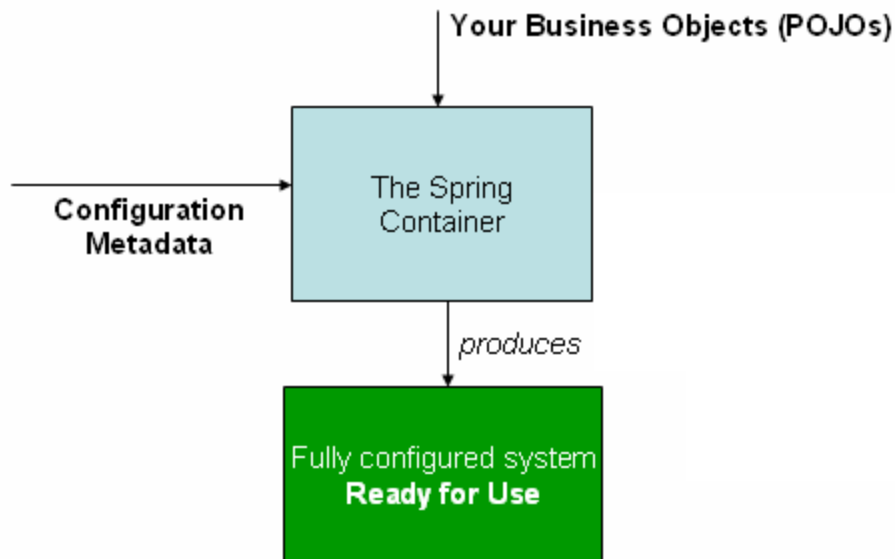
- Stomp and WebSocket integration for real-time messaging in web applications.

Spring Test

- Provides testing support for Spring applications, including unit tests, integration tests, and support for mocking beans.
- **Features:**
 - Mocking Spring beans in unit tests.
 - Integration with JUnit, TestNG.
 - ApplicationContext caching to optimize test execution.

Spring Core

- **Spring IoC (Inversion of Control)**



- is a core feature of the Spring Framework, which facilitates the **loose coupling** of components by injecting dependencies externally, rather than having the components create them internally.
- This approach leads to better flexibility, reusability, and testability of code.

Core Components of Spring IoC Architecture:

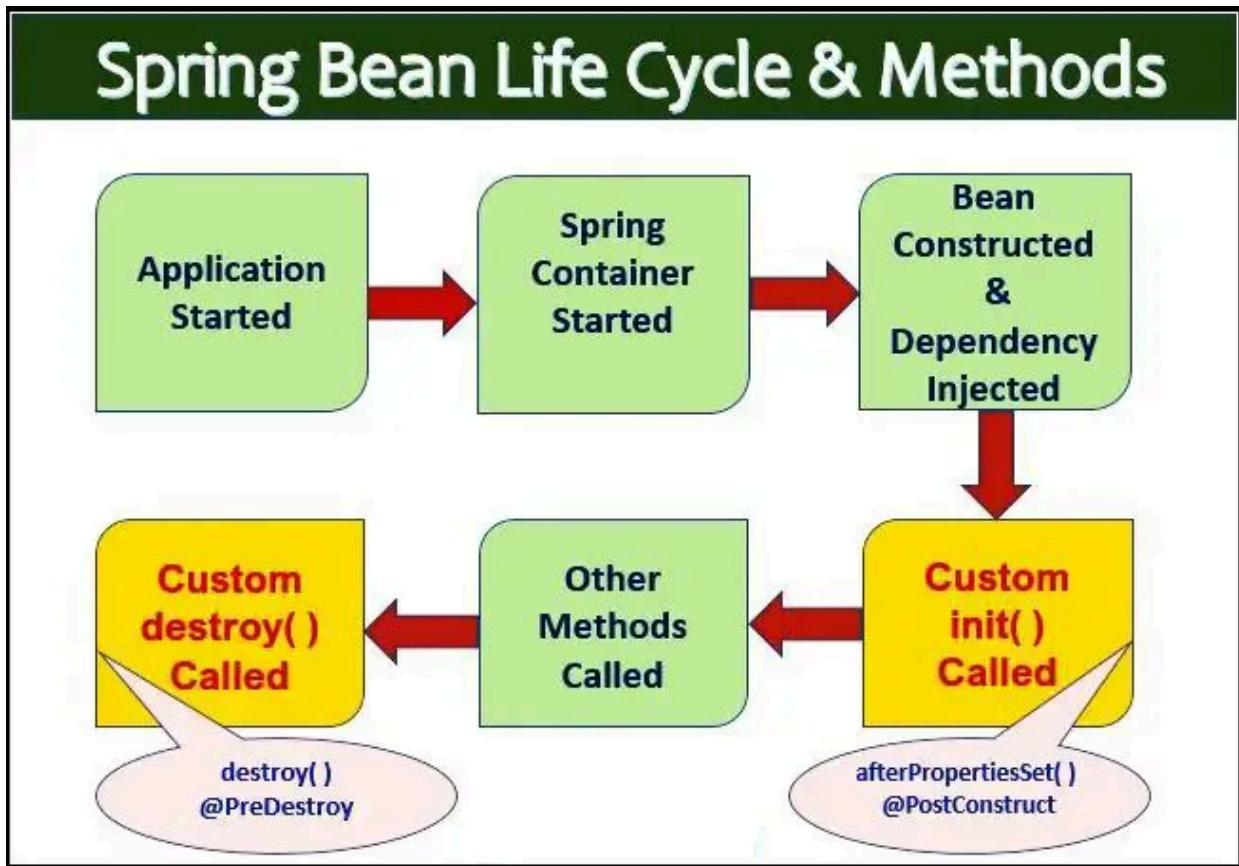
- **Bean**: Any object that is managed by the Spring IoC container.
- **IoC Container**: The heart of the Spring framework, responsible for creating, configuring, and managing beans.
 - **BeanFactory**: The simplest container, provides basic IoC support.
 - **ApplicationContext**: More advanced, provides additional enterprise features like event

propagation, declarative mechanisms to create beans, and AOP integration.

ApplicationContext Implementations:

- Common implementations include:
 - `AnnotationConfigApplicationContext`: For Java-based configuration.
 - `ClassPathXmlApplicationContext`: For XML-based configuration.

Spring Bean Lifecycle:



The Spring IoC container manages the lifecycle of the beans. Below is an overview of the key steps in the Spring bean lifecycle:

1. Bean Instantiation:

- The Spring container instantiates the bean using a constructor or a factory method.

2. Populate Properties (Dependency Injection):

- After the instantiation, Spring injects dependencies (other beans) into the bean either via constructor, setter methods, or field injection.

3. Pre-initialization (BeanPostProcessor):

- The container invokes the `postProcessBeforeInitialization()` method of any beans that implement the `BeanPostProcessor` interface. This allows custom logic to be applied before initialization.

4. InitializingBean Interface / Custom Init Method:

- If the bean implements the `InitializingBean` interface, its `afterPropertiesSet()` method is called.
- Alternatively, you can specify a custom init method to execute after dependency injection and initialization.

5. Bean in Use:

- The bean is fully initialized and is now ready for use by the Spring container or other components of the application.

6. DisposableBean Interface / Custom Destroy Method:

- If the bean implements the `DisposableBean` interface, its `destroy()` method is called when the bean is no longer needed.
- You can also define a custom destroy method to be called when the bean is removed from the container.

What is the Spring IoC container and how does it work?

- The Spring IoC (Inversion of Control) container is responsible for managing the lifecycle and dependencies of beans. It uses dependency injection to manage the dependencies of beans.

Interview Question

Explain the Spring Bean lifecycle in detail.

What are the new features in Spring 6 related to lifecycle management?

- Spring 6 introduces optimizations for GraalVM native images, native Jakarta EE support, and enhanced lifecycle management, making beans more lightweight and performant in cloud-native and modern runtime environments.

How does Spring 6 differ in terms of lifecycle management from earlier versions?

- The lifecycle is fundamentally the same, but Spring 6 enhances startup performance with native image support and integrates better with Jakarta EE APIs. It also drops support for older Java versions, requiring Java 17+.

Can a Spring bean be singleton and prototype at the same time? How does this affect lifecycle?

- A Spring bean cannot be both singleton and prototype. The scope influences the lifecycle; singleton beans are created once per container, while prototype beans are created every time they are requested, meaning their lifecycle ends after initialization.

Bean Naming Convention in Spring:

- **Standard Java Naming:** Bean names in Spring follow the standard Java convention for instance field names.
- **Lowercase Start:** Bean names start with a lowercase letter.
- **Camel Case:** If the name consists of multiple words, they are camel-cased, where the first word starts in lowercase, and subsequent words start with uppercase letters.

Examples:

- `accountManager`
- `accountService`
- `userDao`
- `loginController`

Maven

Maven is a powerful *project management tool* that is based on POM (project object model). It is used for projects build, dependency and documentation.

Understanding the problem without Maven

Creating the right project structure:

Adding set of Jars in each project

Building and Deploying the project:

Dependency injection (DI)

Dependency injection (DI) is a programming technique that helps ensure that objects and functions receive the objects and functions they need to work, instead of creating them internally

XML-Based Configuration

Overview:

- Uses XML files to define beans and their relationships.
- Configuration is done through XML tags in a file typically named `applicationContext.xml` or similar.

```
<!-- Define a bean -->
<bean name="myService" class="com.example.MyService">
  <property name="someDependency" ref="someOtherBean"/>
</bean>
<!-- Another bean definition -->
<bean name="someOtherBean"
class="com.example.SomeOtherBean"/>

<bean id="myService" class="com.example.MyService">
<constructor-arg ref="someOtherBean"/> </bean>
```

<bean>

- **Purpose:** Defines a bean in XML configuration.
- **Usage:** Used to declare and configure beans.

<property>

- **Purpose:** Sets a property of a bean.
- **Usage:** Configures properties of the bean being defined.

<constructor-arg>

- **Purpose:** Defines arguments for a bean's constructor.
- **Usage:** Configures constructor arguments for the bean.

Java-Based Configuration (Class-Based Configuration)

Overview:

- Uses Java classes with annotations to define beans and their relationships.
- Configuration is done through `@Configuration` classes and methods annotated with `@Bean`.

```
@Configuration
@ComponentScan(basePackages = "com.example")
public class AppConfig {
    @Bean
    public MyService myService() {
        return new MyService(someOtherBean());
    }
    @Bean
    public SomeOtherBean someOtherBean() {
        return new SomeOtherBean();
    }
}
```

@Configuration

- **Purpose:** Indicates that a class declares one or more `@Bean` methods.
- **Usage:** Used to define and configure beans in Java-based configuration.

`@Bean`

- **Purpose:** Indicates that a method produces a bean to be managed by the Spring container.
- **Usage:** Used within `@Configuration` classes to define beans.

`@ComponentScan`

Usage: Specifies the packages to scan for annotated components

`@ComponentScan` tells Spring where to look for annotated components (`@Component`, `@Service`, `@Repository`, `@Controller`).

The `basePackages` attribute specifies the packages to be scanned for components.

DI USING Annotation

The `@Value` annotation in Spring is used for injecting values into fields, methods, or constructor arguments. This can be used to inject literal values, expressions, or values from property files.

Injecting Literal Values

Purpose: To inject constant values directly into a field or method.

@Component

```
public class MyService {  
    @Value("Hello, World!")  
    private String greeting;  
    public void printGreeting() {  
        System.out.println(greeting); // Outputs: Hello, World!  
    }  
}
```

Autowiring in Spring Boot:

- **Autowiring:** Spring Boot automatically injects dependencies into Spring beans, reducing the need for manual configuration.
- **@Autowired Annotation:** This annotation is used to mark a field, constructor, or setter method for dependency injection. Spring Boot will then automatically inject the required bean at runtime.

@Qualifier

- **Purpose:** Specifies which bean to inject when multiple beans of the same type are present.
- **Usage:** Used with **@Autowired** to avoid ambiguity in bean injection.

Types of Autowiring:

Field Injection:

```
@Autowired  
private MyService myService;
```

1. Constructor Injection (preferred):

```
@Autowired  
public MyComponent(MyService myService) {  
    this.myService = myService;  
}
```

Setter Injection:

```
@Autowired  
public void setMyService(MyService myService) {  
    this.myService = myService;  
}
```

@Component Annotation:

- **@Component:** This annotation is used to indicate that a class is a Spring-managed component, making it a candidate for component scanning and autowiring.

- When annotated with `@Component`, the class will be automatically detected and registered as a bean during classpath scanning.

Example:

```
@Component
public class MyService {
    // Service logic
}

@Component
public class MyController {
    @Autowired
    private MyService myService; // Dependency injection via
    autowiring

    // Business logic
}
```

Injecting Values from Property Files

Purpose: To inject values from external property files, which allows for more flexible and configurable applications.

Create a Property File (e.g., `application.properties` or `application.yml`):

```
app.name=MyApp  
app.version=1.0
```

```
@Component  
public class AppConfig {  
    @Value("${app.name}")  
    private String appName;  
    @Value("${app.version}")  
    private String appVersion;  
    public void printAppInfo() {  
        System.out.println("App Name: " + appName);    // Outputs: App  
Name: MyApp  
        System.out.println("App Version: " + appVersion); // Outputs: App  
Version: 1.0  
    }  
}
```

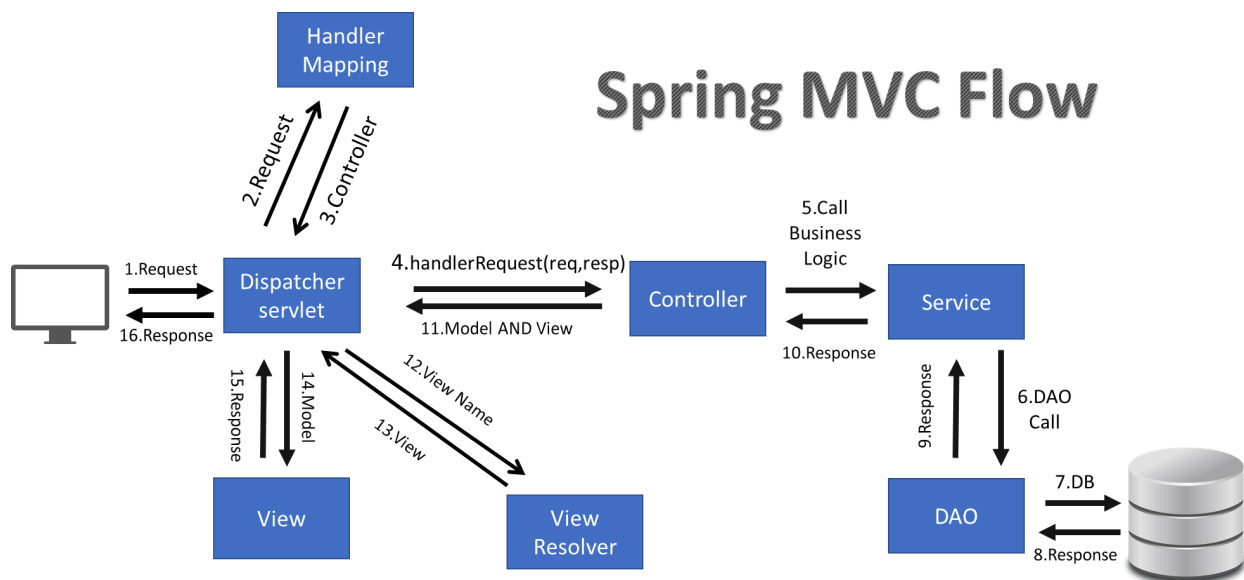
```
@Configuration  
@PropertySource("classpath:application.properties")  
public class AppConfig {  
    // Bean definitions  
}
```

Spring MVC

Spring MVC Architecture

Spring MVC (Model-View-Controller) is a web framework that follows the MVC design pattern. It is a key part of the Spring Framework that helps developers build web applications by separating the logic into three distinct components:

1. **Model:** Represents the application's data or business logic.
2. **View:** Represents the user interface and what the user sees.
3. **Controller:** Acts as an intermediary between Model and View, handling the user input and updating the view accordingly.



Spring MVC Work Flow

- Client(Browser) requests for a Resource in the web Application.
- The Spring front controller i.e, DispatcherServlet first receives the request.
- DispatcherServlet consults the HandlerMapping to identify the particular controller for the given URL.

- HandlerMapping identifies the controller for the given request and sends to the DispatcherServlet.
- DispatcherServlet will call the handleRequest(request,response) method on Controller. A Controller is developed by writing a simple java class which implements Controller interface.
- Controller will call the business method according to business requirement.
- Service class will call the DAO class method for business data.
- DAO interacts with DB to get data.
- DAO returns same data to service.
- Fetched data will be processed according to business requirement and return results to Controller.
- The Controller returns the Model and View in the form of Object back to the Controller i.e, DispatcherServlet.
- The front controller i.e, DispatcherServlet then tries to resolve the actual View which may be JSP,velocity or Free Marker by consulting the View Resolver Object.
- ViewResolver selected view is rendered back to the DispatcherServlet.
- DispatcherServlet consult the particular view with the model.
- View executes and returns HTML output to the DispatcherServlet.
- DispatcherServlet will send the output to the Browser.

Components of Spring MVC

1. DispatcherServlet:

- It is the front controller in the Spring MVC architecture. It intercepts incoming HTTP requests and delegates them to the appropriate controllers for processing.

2. Controller:

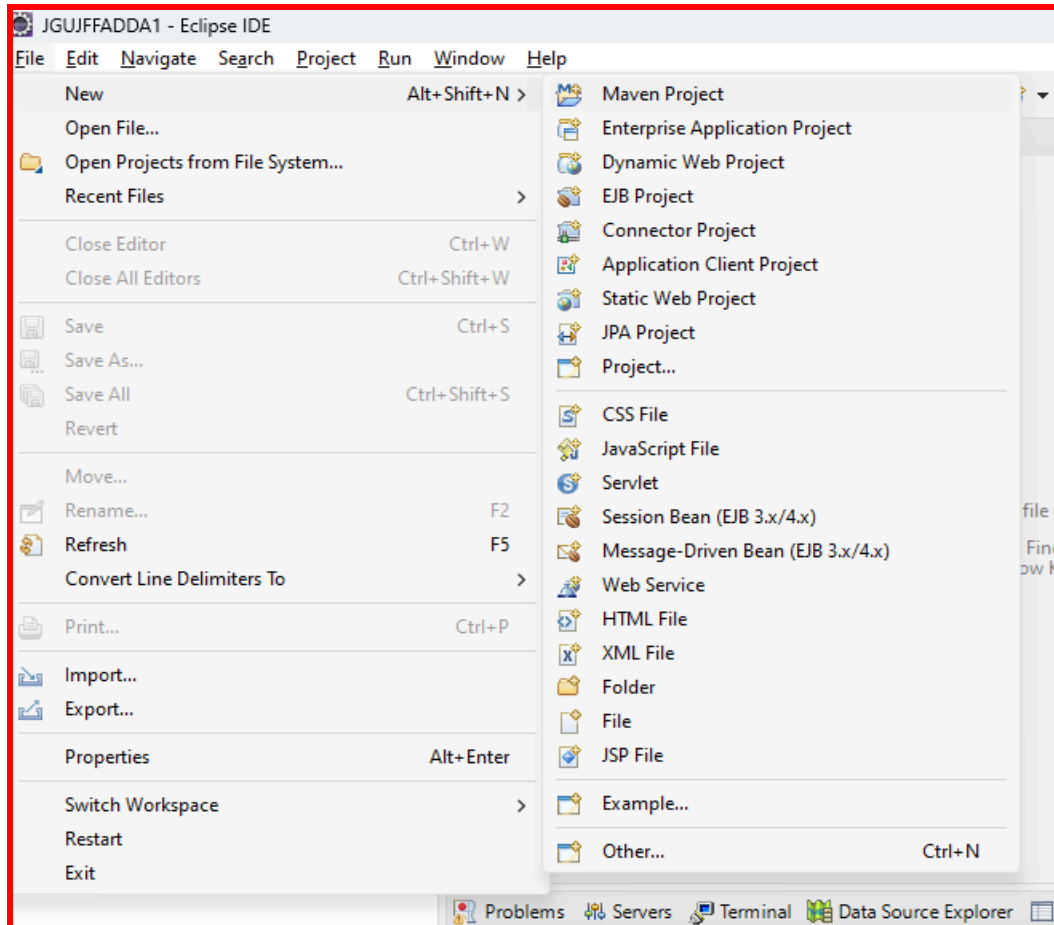
- A controller in Spring MVC is a class that handles the requests coming from the DispatcherServlet. It processes the request, interacts with the model, and then returns a view.

3. Model:

- It contains the business logic and data of the application. The controller interacts with the model to retrieve or manipulate data.
- 4. **View:**
 - The view is responsible for rendering the model data. In a typical Spring MVC application, views are represented by JSP, Thymeleaf, or other templating engines.
- 5. **View Resolver:**
 - This component resolves the view names returned by controllers and maps them to actual view templates (e.g., JSP files).
- 6. **Handler Mapping:**
 - It maps incoming requests to the appropriate controller methods.

Creating Spring MVC project

Step 1 create maven project web archetype 1.4



New Maven Project

New Maven project

Select project name and location

☐ Create a simple project (skip archetype selection)

☒ Use default Workspace location

Location:

Browse...

☐ Add project(s) to working set

Working set:

More...

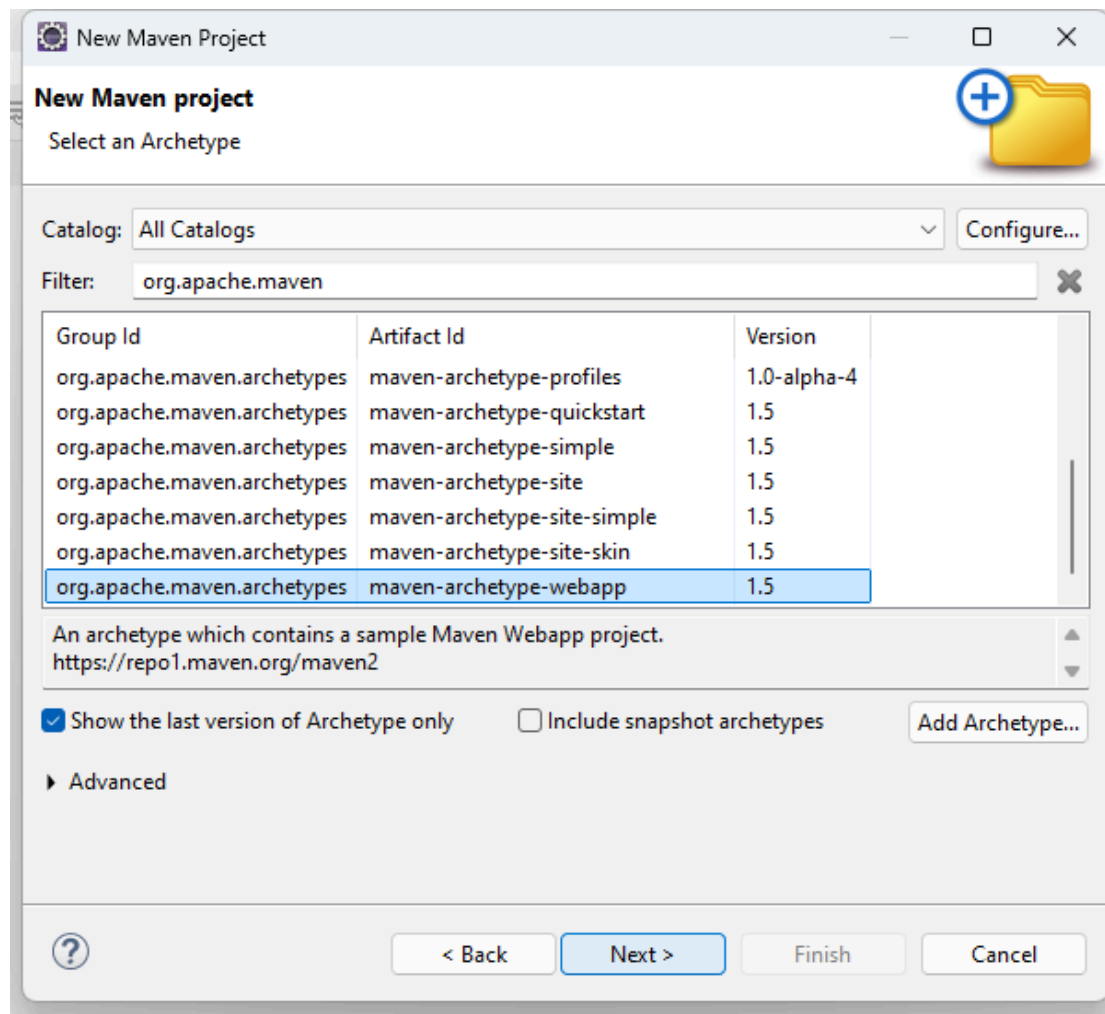
Advanced

< Back

Next >

Finish

Cancel



New Maven Project

New Maven project

Specify Archetype parameters

Group Id:

com.velocitai

Artifact Id:

first-mvc-creation

Version:

0.0.1-SNAPSHOT

Package:

com.velocitai.first_mvc_creation

☒ run archetype generation interactively

Properties available from archetype:

Name	Value

Add...

Remove

Advanced

?

< Back

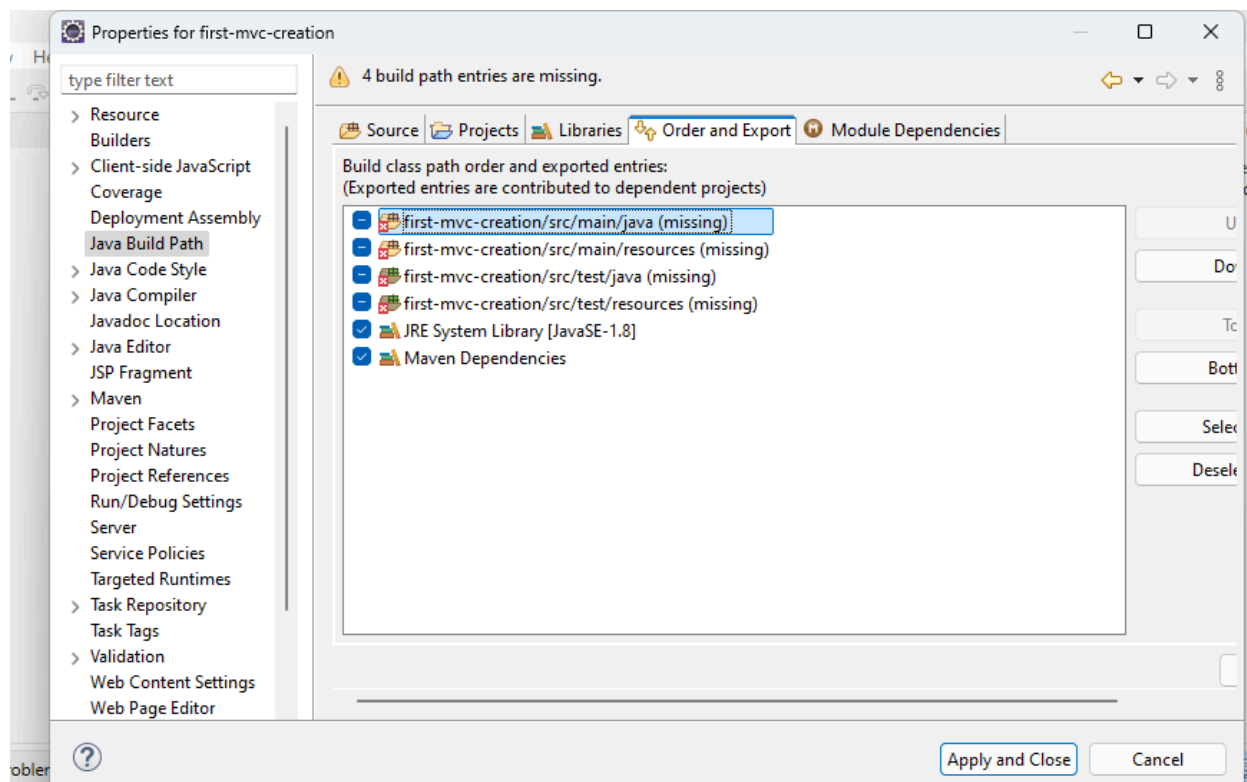
Next >

Finish

Cancel

```
Downloaded from central: f
[INFO] Generating project
[INFO] Archetype repositor
[INFO] Using property: grc
[INFO] Using property: art
[INFO] Using property: ver
[INFO] Using property: pac
Confirm properties config
groupId: com.velocitai
artifactId: first-mvc-crea
version: 0.0.1-SNAPSHOT
package: com.velocitai.fir
Y: : y
```

Enter y and press ok



Add Dependencies


```
<dependencies>
    <dependency>
        <groupId>org.springframework</groupId>
        <artifactId>spring-webmvc</artifactId>
        <version>6.1.12</version>
    </dependency>
    <!--
    https://mvnrepository.com/artifact/jakarta.servlet/jakarta.servlet-api -->
    <dependency>
        <groupId>jakarta.servlet</groupId>
        <artifactId>jakarta.servlet-api</artifactId>
        <version>6.0.0</version>
        <scope>provided</scope>
    </dependency>
</dependencies>
```

Example: Spring MVC Application

Here's an example of a simple Spring MVC application with a controller, model, and view.

```

@Controller
public class UserController {
    @RequestMapping("/user")
    public String getUser(Model model) {
        User user = new User("John", "Doe", 25);
        model.addAttribute("user", user);
        return "userProfile";
    }
}

```

The `@Controller` annotation indicates that the class is a Spring MVC controller. The `@RequestMapping("/user")` maps the URL `/user` to the `getUser()` method, which adds a `User` object to the model. The method returns the view name `"userProfile"`.

2. Model (User.java)

```

public class User {
    private String firstName;
    private String lastName;
    private int age;
    // Constructor, Getters, and Setters
    public User(String firstName, String lastName, int age) {
        this.firstName = firstName;
        this.lastName = lastName;
        this.age = age;
    }
    // Getters and Setters
}

```

Explanation: This is the model class that contains the data. It has three fields: `firstName`, `lastName`, and `age`.

3. **View** (`userProfile.jsp`)

```
<html>
  <body>
    <h1>User Profile</h1>
    <p>First Name: ${user.firstName}</p>
    <p>Last Name: ${user.lastName}</p>
    <p>Age: ${user.age}</p>
  </body>
</html>
```

Explanation: This is a JSP view that displays the user's information. The model attribute `user` is used to render the user's data dynamically.

@Component

- The `@Component` annotation in Spring is used to mark a class as a component, which allows the Spring framework to automatically detect and instantiate the class for dependency injection

- `@Repository`, `@Service`, and `@Controller` are specializations of `@Component` for more specific use cases

- The `@Service` annotation is used to identify classes that provide business logic and functionality.

- The `@Repository` annotation enables a class to perform CRUD operations on a database.

`@CONTROLLER`

- The `@Controller` annotation indicates that a particular class serves the role of a controller.

- Spring does not require you to extend any controller base class or reference the Servlet API.

- The dispatcher scans such annotated classes for mapped methods and detects `@RequestMapping` annotations

`@RequestMapping`

- The `@RequestMapping` annotation is used to map requests to controllers methods.

- You can use it at the class level to express shared mappings or at the method level a specific endpoint mapping.

HTTP Methods and Annotations:

1. **@GetMapping**: Retrieves data from the server (read-only operations).
2. **@PostMapping**: Creates a new resource on the server.
3. **@PutMapping**: Fully updates an existing resource (requires all fields).
4. **@DeleteMapping**: Deletes a resource from the server.
5. **@PatchMapping**: Partially updates an existing resource (updates specific fields only).
- 6.

ModelAndView

•The ModelAndView class in Spring MVC is a key component used to return both **model data** and **view information** from a controller method to the view layer (such as a JSP, Thymeleaf, etc.).

•

```
@Controller
public class MyController {

    @GetMapping("/greeting")
    public ModelAndView showGreeting() {
        ModelAndView modelAndView = new ModelAndView();
        // Setting the view name (the JSP page name or template file)
        modelAndView.setViewName("greeting");
        // Adding data (model) to the view
        modelAndView.addObject("message", "Hello, welcome to Spring
MVC!");
        return modelAndView;
    }
}
```

@RequestParam: Extracts query parameters from the URL.

@PathVariable: Extracts variables from the URI path.

@ModelAttribute: Binds form data to a model object.

@CookieValue: Extracts cookie values from the request.

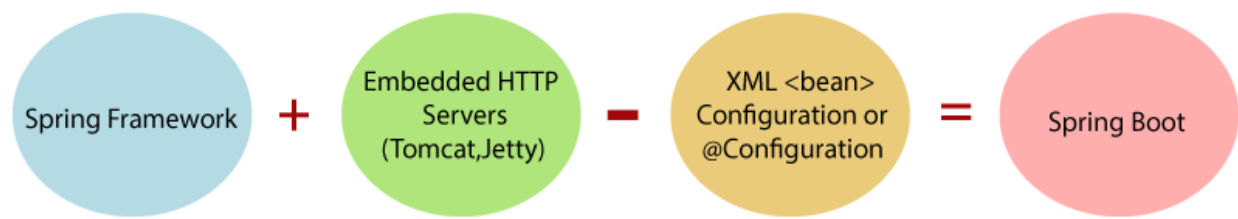
HttpSession: Manages user session information across multiple requests.

Spring form

Spring Boot

Spring Boot is a project that is built on the top of the Spring Framework. It provides an easier and faster way to set up, configure, and run both simple and web-based applications.

Spring Boot is widely used for creating **microservices** and **RESTful web services**, providing an easy way to deploy applications on the cloud and support modern software development practices like CI/CD.



Web services

Web services are standardized ways of enabling communication between different software applications over the internet.

- Designed for application to application interaction
- Not platform independent
- Allow communication over network

Web services are typically used to expose business logic, data, or functionality to other applications via a network (usually the web), in a machine-readable format such as XML or JSON.

SOAP (Simple Object Access Protocol):

- It relies on XML as its messaging format and is known for its strict standards.
- Typically uses HTTP, but can also work over other protocols like SMTP.
- Offers robust security features and is more suited for distributed enterprise environments where reliability and transaction support are important.

REST (Representational State Transfer):

- Uses HTTP methods (GET, POST, PUT, DELETE, etc.) and is lighter than SOAP.
- Most commonly uses JSON or XML for data interchange, but it can also support other formats like plain text.
- **Stateless:** Each request from a client to the server must contain all the information needed to understand and process the request, without relying on stored context on the server.

Why Spring Boot for Web Services?

- **Microservices Architecture:** Many of these companies use Spring Boot because it simplifies the development of microservices, which is critical in modern, scalable, distributed applications.
- **Rapid Development:** Spring Boot accelerates API and web service development through its opinionated approach and ready-to-use configurations.
- **Scalability:** Applications like Netflix, Zalando, and Alibaba deal with massive user bases, and Spring Boot's ability to scale is a major advantage.

Java To JSON Parsing

```
class Person {  
    private String name;  
    private int age;  
    private boolean isEmployed;  
    private double salary;  
    private Gender gender;  
    private List<String> hobbies;  
    private Map<String, Integer> scores;  
}  
//ENUM  
enum Gender {  
    MALE, FEMALE, OTHER  
}
```

```
{  
    "name": "John Doe",  
    "age": 30,  
    "isEmployed": true,  
    "salary": 50000.75,  
    "gender": "MALE",  
    "hobbies": ["Reading", "Swimming",  
    "Coding"],  
    "scores": { "Math": 85, "English": 90,  
    "Science": 88 }  
}
```

ENUM

In Java, **enum** (short for **enumeration**) is a special data type that represents a collection of constants

Spring Boot

Spring Boot is an open-source, Java-based framework that simplifies the development of stand-alone, production-ready Spring applications.

- It is built on top of the **Spring Framework** and offers a way to rapidly develop Java applications with minimal configuration
- Spring Boot focuses on **convention over configuration**, making it much easier and faster for developers to build applications.

Key Features of Spring Boot:

- ☐ Auto-Configuration:
- ☐ Embedded Servers:
- ☐ Convention Over Configuration:

STEPS TO SPRING BOOT PROJECT

Types of Software Architecture

1. Monolithic Architecture:

- **Description:** A single, unified codebase that handles everything in the system.
- **Advantages:**
 - Easier to develop, deploy, and test in small projects.
 - Less overhead in terms of orchestration and communication.
- **Disadvantages:**
 - As the system grows, it becomes hard to manage and scale.
 - A single failure can bring down the entire system.

2. Microservices Architecture:

- **Description:** The application is broken down into independent services that communicate via APIs (e.g., REST or gRPC).
- **Advantages:**
 - Independent deployment and scalability for each service.
 - Fault isolation: failure in one service doesn't affect others.
 - Allows polyglot development (different languages and frameworks).
- **Disadvantages:**
 - More complex to manage.
 - Requires DevOps practices (e.g., Docker, Kubernetes).
 - Communication overhead between services.

1. Core Annotations

- **@Component:** Indicates that a class is a Spring component, allowing it to be automatically detected and registered as a bean in the application context.
- **@Service:** A specialization of **@Component**, used to indicate that a class provides business logic. It typically contains service methods.
- **@Repository:** Another specialization of **@Component**, used for data access layers. It encapsulates database access and handles exceptions.
- **@Controller:** Indicates that a class serves as a controller in the MVC pattern, handling web requests.

- **@Configuration**: Marks a class as a source of bean definitions for the application context. Used for defining beans programmatically.
- **@Bean**: Indicates that a method produces a bean to be managed by the Spring container. Used within **@Configuration** classes.

2. Dependency Injection Annotations

- **@Autowired**: Enables automatic dependency injection. It can be applied to constructors, methods, or fields.
- **@Qualifier**: Specifies which bean to inject when multiple candidates are available. Used in conjunction with **@Autowired**.
- **@Primary**: Indicates that a bean should be given preference when multiple beans of the same type exist.
- **@Value**: Used to inject values into fields from properties files or environment variables.

3. Aspect-Oriented Programming Annotations

- **@Aspect**: Declares a class as an aspect, enabling the definition of cross-cutting concerns (like logging or transactions).
- **@Before**: Indicates that a method should be executed before a join point (e.g., a method execution).
- **@After**: Indicates that a method should be executed after a join point, regardless of the outcome.
- **@Around**: Indicates that a method should be executed around a join point, allowing for pre- and post-processing.
- **@AfterReturning**: Indicates that a method should be executed after a successful join point, allowing access to the return value.

- **@AfterThrowing**: Indicates that a method should be executed if a join point throws an exception, allowing access to the exception.

4. Web Annotations (Spring MVC)

- **@RequestMapping**: Maps HTTP requests to handler methods. It can be used at the class or method level.
- **@GetMapping**: A shorthand for `@RequestMapping(method = RequestMethod.GET)` for handling GET requests.
- **@PostMapping**: A shorthand for `@RequestMapping(method = RequestMethod.POST)` for handling POST requests.
- **@PutMapping**: A shorthand for `@RequestMapping(method = RequestMethod.PUT)` for handling PUT requests.
- **@DeleteMapping**: A shorthand for `@RequestMapping(method = RequestMethod.DELETE)` for handling DELETE requests.
- **@PathVariable**: Binds a method parameter to a URI template variable.
- **@RequestParam**: Binds a method parameter to a query parameter in the request URL.
- **@RequestBody**: Binds the body of the HTTP request to a method parameter, allowing for deserialization of JSON or XML to Java objects.
- **@ResponseBody**: Indicates that the return value of a method should be bound to the web response body, typically for returning JSON or XML.
- **@CookieValue**: Binds a method parameter to a cookie value in the request.
- **@SessionAttributes**: Used in a controller to indicate which attributes should be stored in the session.

5. Testing Annotations

- **@SpringBootTest**: Used to create an application context for integration tests in Spring Boot.
- **@MockBean**: Creates a mock of a bean for testing purposes and injects it into the application context.
- **@Test**: Marks a method as a test method, used in conjunction with testing frameworks like JUnit.

6. Spring Boot Specific Annotations

- **@EnableAutoConfiguration**: Enables Spring Boot's auto-configuration feature, which automatically configures beans based on the classpath settings.
- **@SpringBootApplication**: A convenience annotation that combines **@Configuration**, **@EnableAutoConfiguration**, and **@ComponentScan**.
- **@PropertySource**: Specifies the property files to be loaded by the Spring environment.

7. Validation Annotations

- **@Valid**: Triggers validation of a method argument, typically used in conjunction with Spring's validation framework.
- **@Validated**: Indicates that a class or method should be validated.

8. Others

- **@Scope**: Specifies the scope of a bean, such as singleton or prototype.
- **@Lazy**: Indicates that a bean should be lazily initialized, i.e., created only when it is needed.

